

Model Verification Testing NetLogo Code

EES 4760/5760

Agent-Based and Individual-Based Computational Modeling

J. Magnolia Gilligan



Getting Started

Announcement

- I will be traveling on Oct. 6–8 and will have to miss class
 - On Tuesday we can discuss options for making up the missed classes.

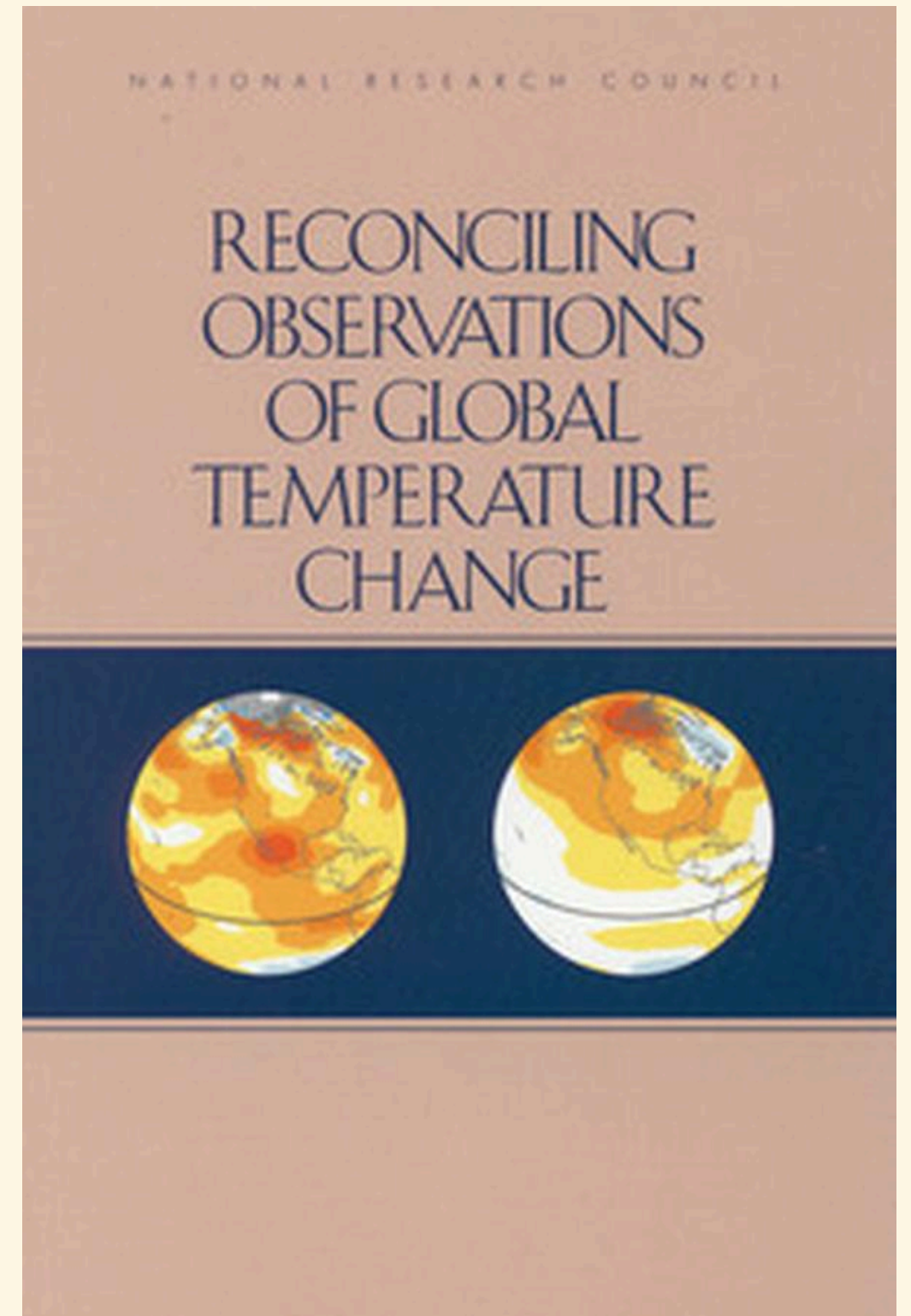
Download Files

- Download files from Download page for “Models for Testing and Verification” on the main downloads page.
 - <https://www.ees4760.jmgilligan.org/downloads/07-model-testing-verification/>

Software Testing

Errors in Programming

- Do you trust your model?
- All computer code has errors
 - Empirical studies:
 - Professional software engineers make ~1 error per 100 lines of code
 - Some errors matter more than others
- Software engineering has developed systematic ways to prevent, find, and fix errors



National Academies: *Reconciling Observations of Global Temperature Change* (2000)

Testing Agent-Based Models

Agent-Based and Individual-Based Modeling A PRACTICAL INTRODUCTION SECOND EDITION



Steven F. Railsback and Volker Grimm

Ecological Modelling 520 (2026) 111669



ELSEVIER

Contents lists available at ScienceDirect

Ecological Modelling

journal homepage: www.elsevier.com/locate/ecolmodel



Testing NetLogo model code

Steven F. Railsback^a, Jocelyn Bliven^b, Volker Grimm^{c,*}, Jacob Kelter^d

^a Lang Railsback & Associates, 250 California Avenue, Arcata, CA, 95521 USA
^b QSIDE Institute, qsideinstitute.org, Williamstown, MA, 01267 USA
^c Department of Ecological Modelling, Helmholtz Centre for Environmental Research – UFZ, Permoserstraße 15, 04318 Leipzig, Germany
^d NetLogo Center, Northwestern University, Evanston, IL, 60208 USA

ARTICLE INFO

Keywords:
Agent-based modeling
Code testing
Individual-based modeling
NetLogo
Software
Verification

ABSTRACT

In simulation modeling, good code testing practices provide (a) efficiency, by reducing time wasted using erroneous software; (b) credibility, by providing evidence that important errors are unlikely; and (c) understanding, by showing how complex model results arose. However, code testing receives little attention in agent-based modeling, and many modeling projects have been derailed by undetected code errors. We present code testing methods appropriate for models implemented in NetLogo by novice programmers. An efficient cycle for implementing new models includes: drafting a written model description and testing the “submodels” that each represent a separate process, coding the model while updating the written description to correct ambiguities and errors, and then thorough code testing. As the model is then used and revised, the written description, code, and code tests are all updated. We identify eight kinds of errors that are especially common in NetLogo software. Logic errors are the hardest to detect; doing so typically requires producing detailed test output for key submodels and then analyzing that output for unexpected results. We provide methods for producing test output, with example NetLogo code. Test output can be analyzed graphically, statistically, by tracing the fate of individual agents, or by comparing it to an independent implementation of the submodel. When unexpected results are found, a variety of “debugging” methods help determine whether they are caused by software errors and, if so, where the errors are. Documenting code testing methods and results is also key to model credibility and efficiency: documented tests provide evidence that the code is reasonably error-free and makes it easier to repeat tests as models are revised. While NetLogo currently lacks code-testing tools such as stepwise debuggers and unit testers, its powerful commands make it easy to add similar capabilities to models.

1. Introduction

Good code testing practices provide efficiency and credibility to the process of developing and using a simulation model, and therefore are a major concern in professional software development. Testing code throughout the modeling process has multiple benefits (considered more fully at Sect. 3, below), especially avoiding time wasted trying to understand and validate erroneous code and contributing to production of a complete and accurate model description. And, of course, being able to demonstrate that our tools do what we say they do—and that we know exactly what they do—is essential to credible science.

The benefits of code testing are even more important as we increasingly rely on artificial intelligence (AI) to produce model code. When a modeler uses AI to assist with, or entirely take over, code

production, the code *may* be less likely to have errors but we cannot assume it is error-free. Just as important, we cannot assume that AI will identify and ask the modeller to resolve the inevitable ambiguities and errors in the model description that the AI is trying to implement, instead of simply making something up. Code testing of the kind we address here is essential for making sure AI-generated code is accurate and documented in sufficient detail.

Code testing seems especially important for mechanistic and agent-based models (ABMs), in part because they typically require larger and more complex programs. For simpler models that are more dependent on parameters fit to observations, minor programming errors may be compensated for and hidden by the calibration process. Mechanistic models, however, often include a variety of submodels for different mechanisms, only some of which might strongly affect results under

* Corresponding author.
E-mail addresses: Steve@LangRailsback.com (S.F. Railsback), jocelyn.bliven@gmail.com (J. Bliven), volker.grimm@ufz.de (V. Grimm), jacobkelter@northwestern.edu (J. Kelter).

<https://doi.org/10.1016/j.ecolmodel.2026.111669>
Received 10 April 2026; Received in revised form 13 May 2026; Accepted 15 May 2026
Available online 20 June 2026

Aspects of Testing

- Basic bug hunting:
 - Making sure the code compiles and runs
- Software verification:
 - Does the code do what the written specification says?
- Software validation:
 - Does the software satisfy the requirements?
 - Does the model accurately represent the system it's simulating?

Polish Software-Testing Championship, 2016



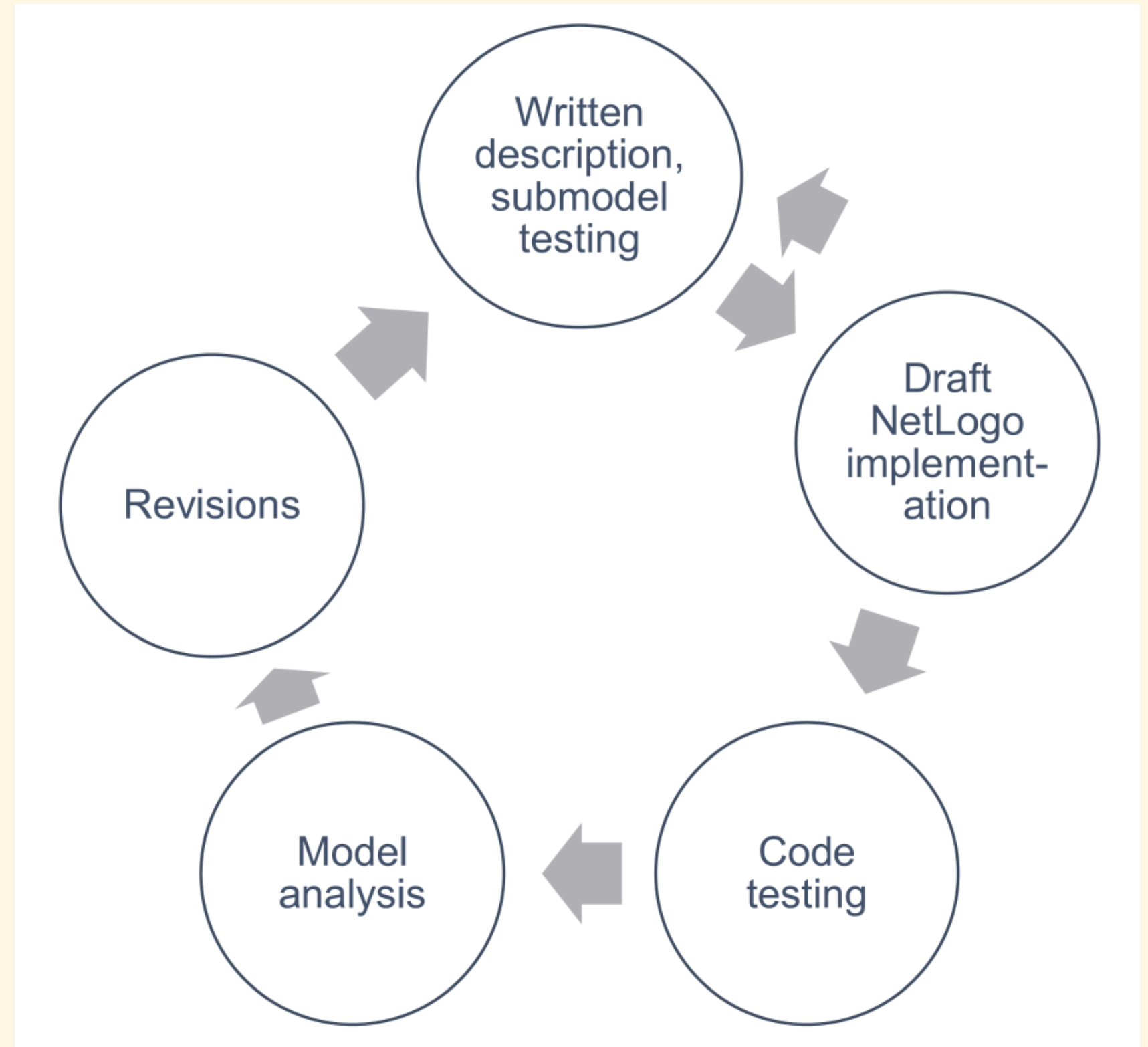
Photo: Radoslaw Smilgin/Wikimedia

Verification of Agent-Based Models

Good Practices in NetLogo

- Start with a solid written specification (ODD)
 - Don't go overboard
 - Develop specification iteratively with coding, testing, etc.
- Bite-sized pieces:
 - Implement and test submodels separately
 - Independent implementation (spreadsheet, Python, R, Matlab, etc.)
- Then build the complete model

NetLogo Modeling Cycle

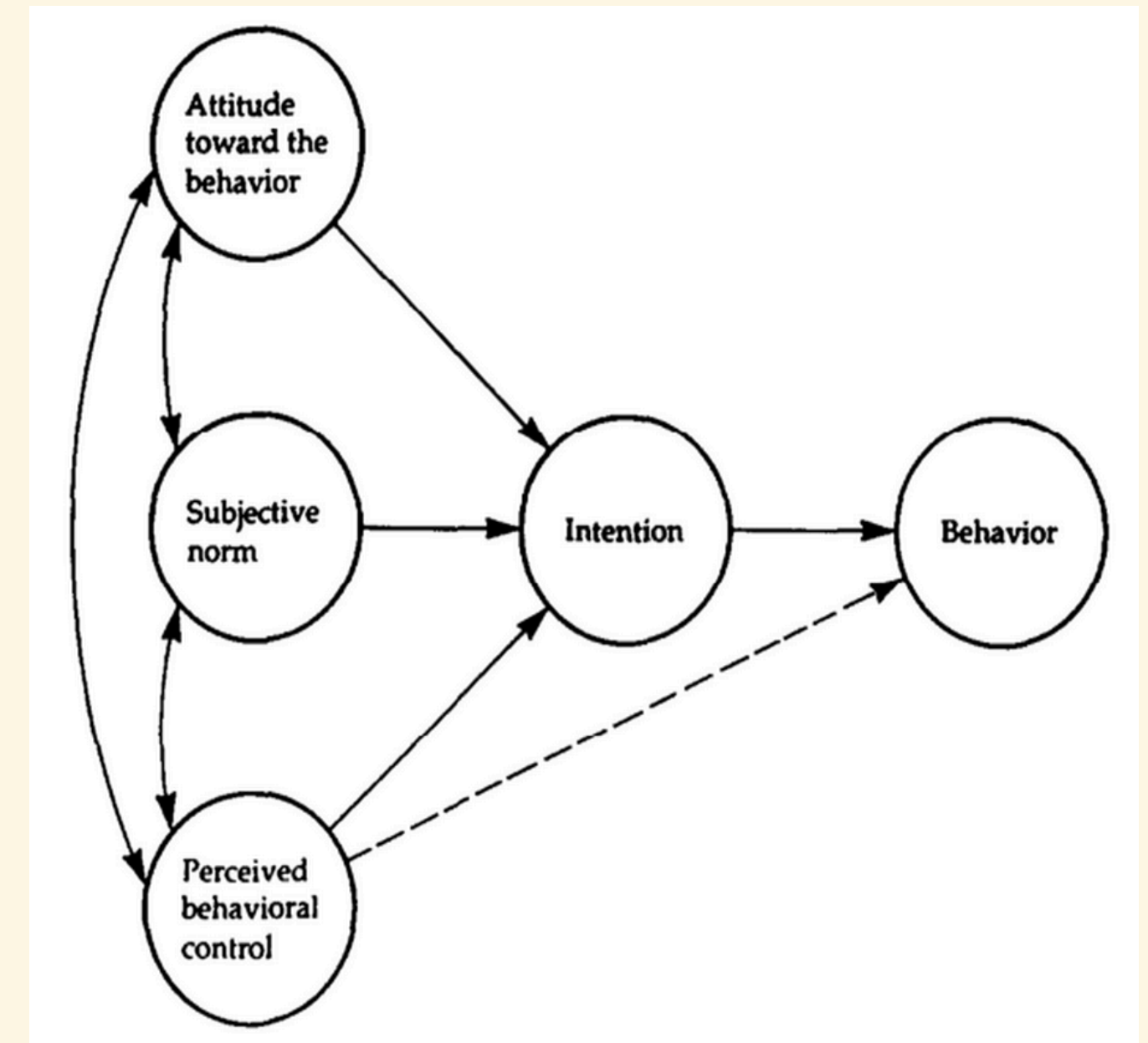


Validation, Verification, and Ambiguous Theory

- Theory of Planned Behavior (Ajzen 1985)
 - Widely used theory of human decision-making
 - Turning schematics into equations & code is ambiguous
- Muelder & Filatova (2018):
- Agent-based model of household solar energy adoption
 - 3 different implementations of the theory of planned behavior
 - Wildly different results:
 - Fraction of households adopting solar power ranged from 58% to 91%
- Testing isn't only about finding errors:
 - It's also about understanding your modeling choices



H, Muelder, & T. Filatova, J. Artificial Societies & Social Simulation **21**, 5 (2018).
DOI: 10.18564/jasss.3855



I. Ajzen, *Org. Behavior & Human Decision Processes* **50**, 179 (1991)
DOI: 10.1016/0749-5978(91)90020-t

Finding Errors in NetLogo Models

Different Kinds of Errors

1. Syntax mistakes
2. Cut-and-paste errors
3. Context errors
4. Misunderstanding the language
5. Incorrect world settings
6. Run-time errors
7. Scale-conversion errors
8. Logic errors

Different Kinds of Errors

1. Syntax mistakes
2. Cut-and-paste errors
3. Context errors
4. Misunderstanding the language
5. Incorrect world settings
6. Run-time errors
7. Scale-conversion errors
8. Logic errors

- Syntax mistakes

```
set age age+1
```

```
let partner one-of other turtles
```

```
let prospects other [turtles in-radius 5]  
of partner
```

```
let prospects [other turtles in-radius 3]  
of partner
```

Different Kinds of Errors

1. Syntax mistakes
2. Cut-and-paste errors
3. Context errors
4. Misunderstanding the language
5. Incorrect world settings
6. Run-time errors
7. Scale-conversion errors
8. Logic errors

- Cut-and-paste errors

```
ask turtles [  
    set xcor random-normal 0  
    (world-width / 3)  
]  
  
ask turtles [  
    set xcor random-normal 0  
    (world-width / 3)  
    set ycor random-normal 0  
    (world-width / 3)  
]
```

Different Kinds of Errors

1. Syntax mistakes
 2. Cut-and-paste errors
 3. Context errors
 4. Misunderstanding the language
 5. Incorrect world settings
 6. Run-time errors
 7. Scale-conversion errors
 8. Logic errors
- Context errors
 - Asking turtles to do patch things or vice-versa
 - Asking one breed of turtle to do things appropriate to a different breed
 - Subtle context errors:
 - Confusion about `self` vs. `myself`

Different Kinds of Errors

1. Syntax mistakes
2. Cut-and-paste errors
3. Context errors
4. Misunderstanding the language
5. Incorrect world settings
6. Run-time errors
7. Scale-conversion errors
8. Logic errors

- Misunderstanding primitives

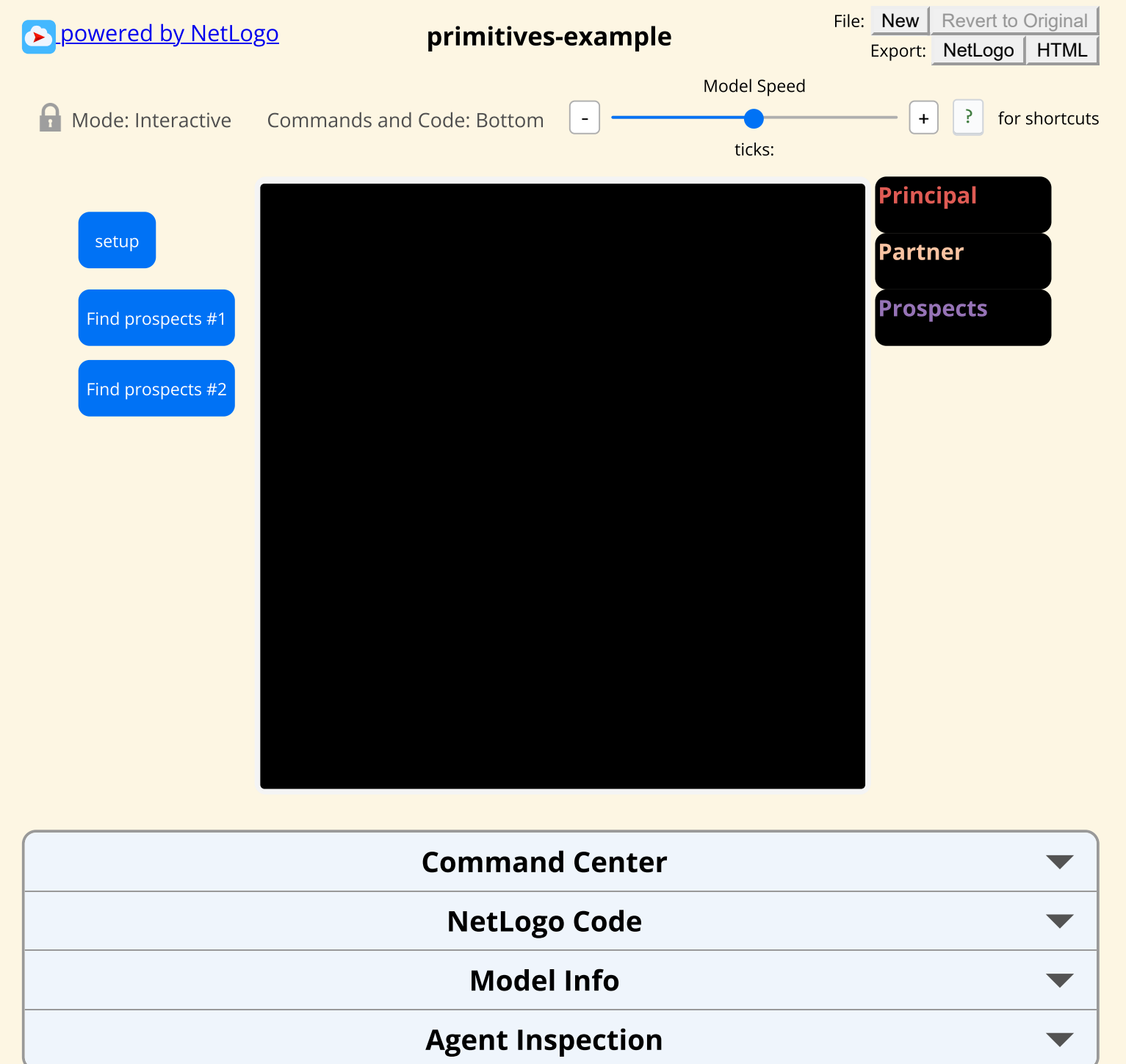
```
if prob < random 1.0  
if prob < random-float 1.0
```

- Incorrect world settings:
 - To wrap or not?
 - Location of origin

Different Kinds of Errors

1. Syntax mistakes
2. Cut-and-paste errors
3. Context errors
4. Misunderstanding the language
5. Incorrect world settings
6. Run-time errors
7. Scale-conversion errors
8. Logic errors

- Open `primitives-example.nlogo`



Different Kinds of Errors

1. Syntax mistakes
2. Cut-and-paste errors
3. Context errors
4. Misunderstanding the language
5. Incorrect world settings
6. Run-time errors
7. Scale-conversion errors
8. Logic errors

- Open `modified-flocking.nlogo`

The screenshot shows the NetLogo interface for the 'modified-flocking' model. At the top, there's a 'powered by NetLogo' logo and the model name 'modified-flocking'. To the right are buttons for 'File: New', 'Revert to Original', 'Export: NetLogo', and 'HTML'. Below this is a 'Model Speed' slider with a 'ticks:' label and a '?' icon for shortcuts. The main control panel on the left includes a 'population' slider set to 300, a 'setup' button, and a 'go' button. Below these are sliders for 'vision' (5 patches), 'minimum-separation' (1 patches), 'max-align-turn' (5 degrees), 'max-cohere-turn' (3 degrees), and 'max-separate-turn' (1.5 degrees). At the bottom of this panel is a 'vision-algorithm' dropdown menu set to 'circle'. The main display area is a large black rectangle. At the bottom of the interface is a 'Command Center' panel with tabs for 'Command Center', 'NetLogo Code', and 'Model Info', each with a dropdown arrow.

Different Kinds of Errors

1. Syntax mistakes
2. Cut-and-paste errors
3. Context errors
4. Misunderstanding the language
5. Incorrect world settings
6. Run-time errors
7. Scale-conversion errors
8. Logic errors

- Open `modified-flocking-fixed.nlogo`

The screenshot shows the NetLogo interface for the 'modified-flocking' model. At the top, there's a 'powered by NetLogo' logo and a title bar with 'modified-flocking'. Below the title bar, there are file management buttons: 'File: New', 'Revert to Original', 'Export: NetLogo', and 'HTML'. A 'Model Speed' slider is set to a middle position, with a 'ticks:' label below it. To the left of the slider, there's a 'Mode: Interactive' label and 'Commands and Code: Bottom' text. Below these are several sliders for model parameters: 'population' (set to 300), 'vision' (set to 5 patches), 'minimum-separation' (set to 1 patch), 'max-align-turn' (set to 5 degrees), 'max-cohere-turn' (set to 3 degrees), and 'max-separate-turn' (set to 1.5 degrees). There's also a 'vision-algorithm' dropdown menu set to 'circle'. A 'setup' button and a 'go' button are located below the sliders. The main area is a large black rectangle representing the model's world. At the bottom, there's a 'Command Center' section with three tabs: 'Command Center', 'NetLogo Code', and 'Model Info', each with a dropdown arrow.

Different Kinds of Errors

1. Syntax mistakes
2. Cut-and-paste errors
3. Context errors
4. Misunderstanding the language
5. Incorrect world settings
6. Run-time errors
7. Scale-conversion errors
8. Logic errors

- Runtime errors

- Syntax is correct, but computer can't execute:

```
show exp 1000  
show sqrt -1  
show 1 / 0
```

- Watch out for random-normal, which can sometimes produce much larger or smaller values than you expect

```
show min (n-values 1E5  
[ -> random-normal 10 2.5 ])
```

Different Kinds of Errors

1. Syntax mistakes
 2. Cut-and-paste errors
 3. Context errors
 4. Misunderstanding the language
 5. Incorrect world settings
 6. Run-time errors
 7. Scale-conversion errors
 8. Logic errors
- Scale-conversion errors
 - NetLogo thinks in terms of ticks and patches
 - If you're representing real-world units (meters, days, etc) ...
 - Be careful to be consistent in converting from
 - Ticks to real time
 - Patches to real lengths

Logic Errors

- The previous errors are more straightforward to find and fix
- Logic errors are subtle, elusive, and difficult.
- Finding logic errors:
 - Producing test output:
 1. Interactive viewing
 2. Events output
 3. Procedure monitor
 4. Procedure explorer
 5. Defensive programming
 - Unit testing
 - Regression testing
 - Analyzing test output:
 - Graphical analysis
 - Statistical analysis
 - Execution tracing
 - Independent implementation

Interactive Viewing

- Open `butterflies.nlogo` in `butterfly-model` folder
 - Process:
 - Each tick:
 - Probability q that butterfly moves to the highest neighboring patch
 - Otherwise it moves to a random neighbor patch

The screenshot shows the NetLogo 'butterflies' model interface. At the top, it says 'powered by NetLogo' and 'butterflies'. The top right has file management buttons: 'File: New', 'Revert to Original', 'Export: NetLogo', and 'HTML'. Below this is a 'Model Speed' slider with a 'ticks:' label and a 'for shortcuts' button. The main interface is divided into a control panel on the left and a large black viewing area on the right. The control panel includes buttons for 'setup', 'go' (with a green flag icon), and 'one step'. There is a 'concentrate-turtles' toggle switch. A slider for 'q' is set to 0.4, with an 'Increment q' button below it. At the bottom of the control panel are two toggle switches: 'leave-trails?' and 'color-visited?', each with a corresponding action button ('Erase trails' and 'Recolor patches' respectively). The bottom of the interface features a 'Command Center' and a list of expandable sections: 'NetLogo Code', 'Model Info', and 'Agent Inspection'.

Testing Butterfly Movement

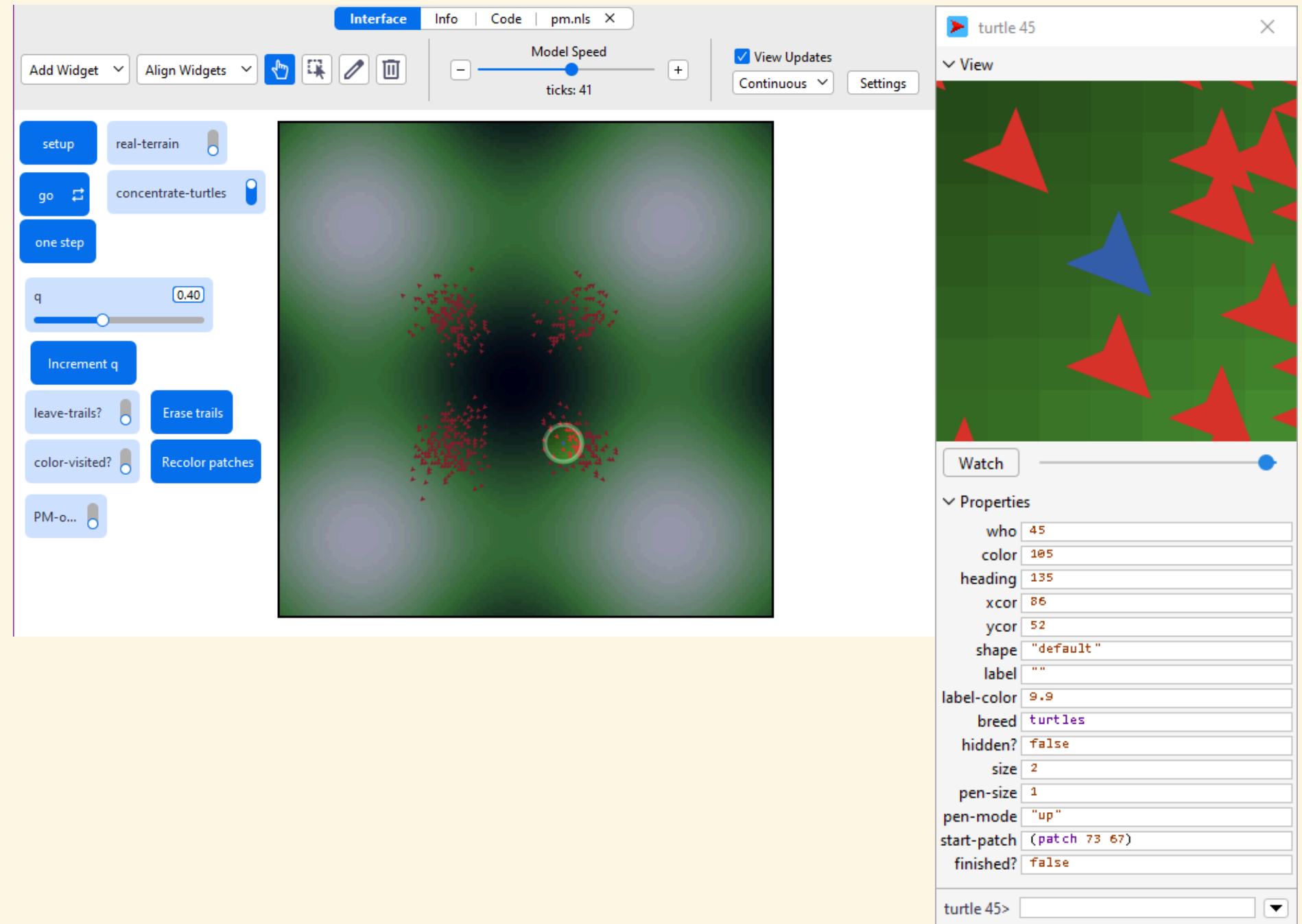
- Watching hundreds of turtles move is hard to interpret
 - Focus on one turtle
 - Take single steps
- Primitives
 - `inspect`
 - `watch`
 - `follow`
- Challenge:
 - Movement is stochastic
 - Randomness is hard to evaluate by eye



The image shows the NetLogo interface for a model titled "butterflies". At the top, there's a "powered by NetLogo" logo and a "File:" menu with options "New" and "Revert to Original". Below that, an "Export:" menu shows "NetLogo" and "HTML". A "Model Speed" slider is set to a middle position, with a "ticks:" label. The interface is in "Mode: Interactive" with "Commands and Code: Bottom". On the left, there's a command palette with buttons for "setup", "go" (with a green flag icon), "one step", and a slider for "q" set to 0.4. Below the "q" slider is an "Increment q" button. There are also two toggle switches: "leave-trails?" (off) and "color-visited?" (off). To the right of these are buttons for "concentrate-turtles" (with a toggle switch), "Erase trails", and "Recolor patches". The main view is a large black rectangle. At the bottom, there's a panel with four expandable sections: "Command Center", "NetLogo Code", "Model Info", and "Agent Inspection", each with a downward arrow.

Testing Butterfly Movement

- Watching hundreds of turtles move is hard to interpret
 - Focus on one turtle
 - Take single steps
- Primitives
 - `inspect`
 - `watch`
 - `follow`
- Challenge:
 - Movement is stochastic
 - Randomness is hard to evaluate by eye



Systematically capture output for analysis

1. Events output

- Manually write data to file

2. Procedure monitor

- Helper functions to save output from one procedure at a time
- Code is in `butterfly-PM.nlogox`

3. Procedure explorer

- Write a test function to repeatedly execute a single procedure with a wide range of conditions or inputs

- Procedure Monitor

```
__includes ["pm.nls" ]

globals [
  ...
]
...
to setup
  ca

  set PM-on? True
  PM-setup "PM-output.csv" (list
    "turtle" "xcor" "ycor" "elevation"
    "q" "random"
    "nbor-x" "nbor-y" "nbor-elevation"
    "uphill?" "moved?"
    "final-xcor" "final-ycor"
    "final-elevation")

  ...
end
```

Procedure Monitor Details

```
to go ; This is the master schedule
```

```
  ask turtles with [not finished?] [move]
```

```
  tick
```

```
  ;; At end of tick,  
  ;; write procedure monitor output to file.
```

```
  PM-write-output
```

```
  ; stop when all the butterflies are at the summit,  
  ; 1000 ticks, whichever comes first
```

```
  if ticks >= 1000 or all? turtles [finished?]
```

```
  [  
    stop  
  ]
```

```
end ; of go
```

```
to move
```

```
  let rand random-float 1.0
```

```
  let old-patch patch-here
```

```
  PM-add (list who xcor ycor elevation q rand)
```

```
  PM-add [ (list pxcor pycor elevation) ] of  
          max-one-of neighbors [elevation]
```

```
  let uphill? false
```

```
  ifelse random-float 1 < q
```

```
  [  
    set uphill? true  
    uphill elevation
```

```
  ] ; Move uphill
```

```
  [  
    move-to one-of neighbors  
  ] ; Otherwise move randomly
```

```
  PM-add (list uphill? (old-patch != patch-here)  
          xcor ycor elevation)
```

```
  set visited? true
```

```
  if color-visited? [ set pcolor yellow ]
```

```
  PM-save-row
```

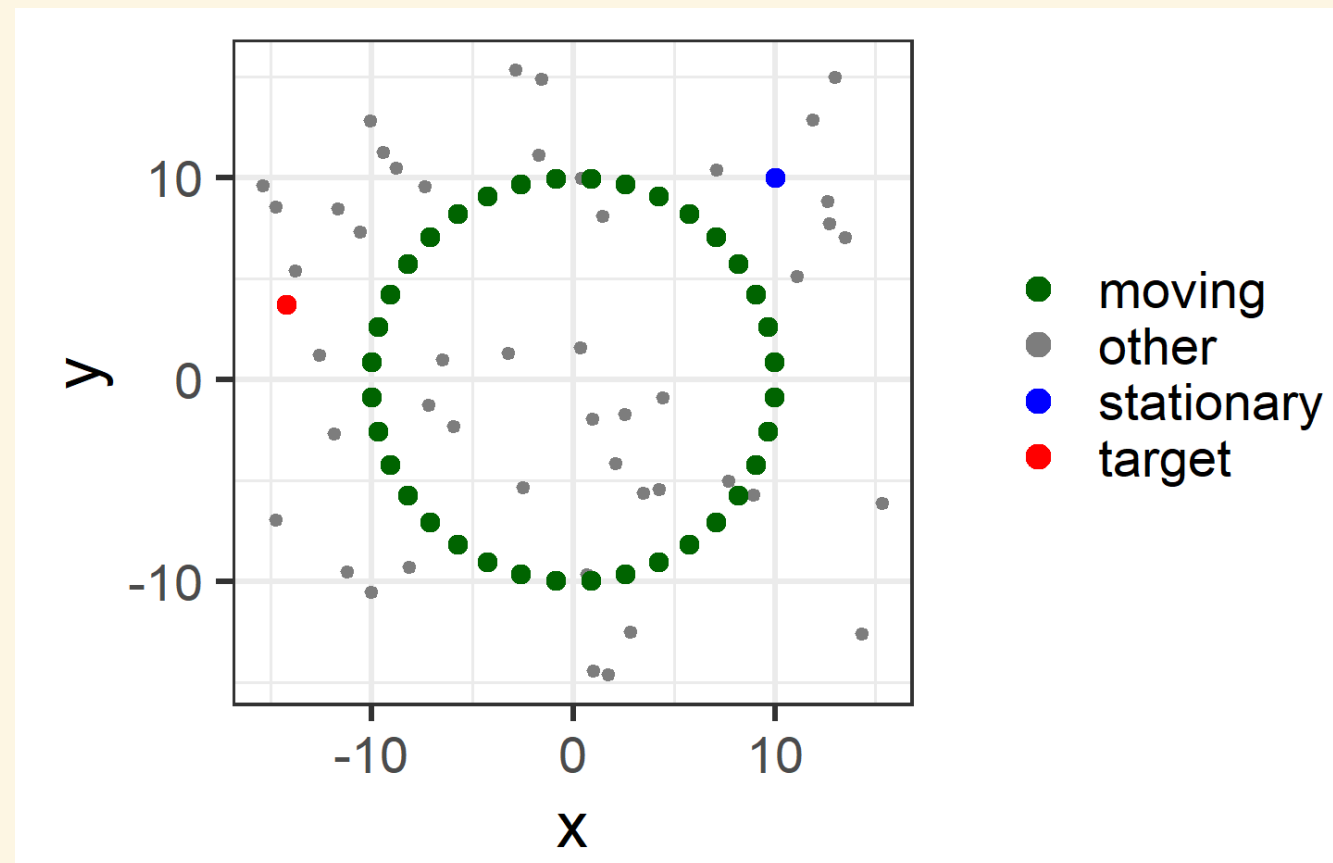
```
end ; of move procedure
```

Procedure Explorers

- In a complete model, many things are happening
- It can be hard to catch and identify infrequent anomalies or errors
- Instead of running the whole model, write a procedure that tests a single procedure intensively
- Open file `flocking_PE.nlogo`
- Setup and run for a while
- Stop and type “explore-headings” in the observer

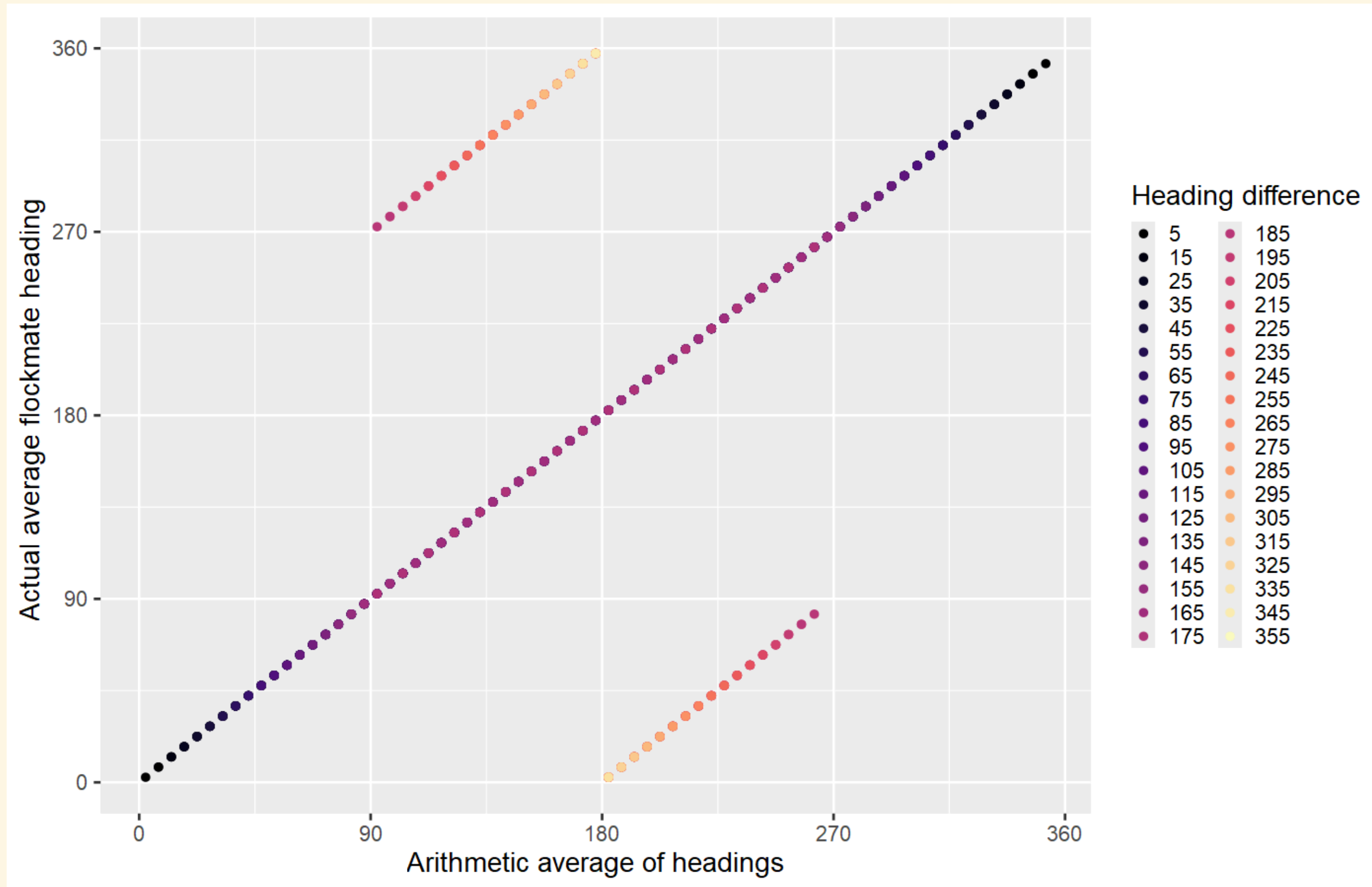
Procedure Explorer Function

- `explore-headings` procedure:
 - Picks a “test turtle” at random
 - Assigns two flockmates at random
 - One stationary flockmate
 - Positioned at 10,10
 - One moving flockmate



- Stationary flockmate varies heading, but is always at 10,10
- Moving flockmate goes to 0,0 and moves forward 10 in the direction it's heading
- Record:
 - Positions
 - Headings
 - Calculated:
 - Average heading of flockmates (stationary and moving)
 - Heading for target to fly to average position of flockmates

Graphical Analysis of Output



Defensive Programming

- Unit tests:
 - Before you write a procedure or reporter, design tests to make sure it does what you intend
 - Execute tests during model runs
 - Consistency checks for agents:
 - Each turtle's turtles-own variables should always be in a consistent state
 - Same for patches, links, etc.
- “Testing is fun”
 - Written by Pierre-Olivier Chasset
 - Based on testthat package for R, by Hadley Wickham
 - Modified by me
 - <https://github.com/j-magnolia/tif>
 - tif.nls in the downloaded files
- Open the file
`butterflies_tif.nlogox`

Using TIF

- Put the line

```
__includes[ "tif.nls" ]
```

near the top

- Put the line

```
initialize-tests
```

in to setup

- To start a group of tests:

```
set-context "context description"
```

- To report on the control center after performing a set of tests:

```
resume-all-tests
```

- For each test in the context:

```
test-that "name of test"  
expect-that <expression> <condition>
```

- Example

```
test-that "turtle is happy"  
expect-that happy? is-true  
test-that "hope is greater than fear"  
expect-that hope is-greater-than fear
```

Using Unit Testing

- Unit testing is valuable, but it can produce a lot of output to the command center
- You may want to put the unit testing into test procedures rather than your main model code
- Alternatively, have a global variable like testing? and only run the unit tests if testing? is true

```
All tests (32117) passed. Congratulations.  
Testing butterfly movement: tick 0
```

```
All tests (707) passed. Congratulations.  
Testing butterfly movement: tick 1
```

```
All tests (1419) passed. Congratulations.  
Testing butterfly movement: tick 2
```

```
...  
Testing butterfly movement: tick 78
```

```
All tests (55325) passed. Congratulations.  
Testing butterfly movement: tick 79
```

```
  **FAILS** Test that butterfly 115 moved,  
            test #1: did not expect (patch 118 118)  
            but got it.
```

```
  **FAILS** Test that butterfly 115 moved to highest neighbor,  
            test #1: expected (patch 119 118) and got  
            (patch 118 118) instead.
```

```
Tests finished with errors: 2/56033 tests failed!  
Testing butterfly movement: tick 80
```

```
  **FAILS** Test that butterfly 227 moved,  
            test #1: did not expect (patch 118 118)  
            but got it.
```

```
  **FAILS** Test that butterfly 227 moved to highest neighbor,  
            test #1: expected (patch 118 119) and  
            got (patch 118 118) instead.
```

```
Tests finished with errors: 4/56730 tests failed!
```

Fixing Errors

- We occasionally see two errors:
 - The butterfly is supposed to move to a new patch every tick, but it didn't move
 - This only happens when the butterfly is trying to move uphill, and it doesn't move to the highest neighbor
- Why?

```
to move
  let rand random-float 1
  let old-patch patch-here

  let uphill? false

  ifelse random-float 1.0 < q
    [
      set uphill? true
      uphill elevation
    ] ; Move uphill
    [
      move-to one-of neighbors
    ] ; Otherwise move randomly
  ...
end
```

Regression Testing

- Find an error and fix it
 - As you continue to work on the model, make sure you don't change something that brings the error back (regression)
- Every time you find an error and fix it, write a test
- Maintain a complete suite of tests to make sure that errors stay fixed
- Regression tests should be run separately from running the model for research.
- Run the complete suite of regression tests at every major milestone:
 - A new public release
 - Making a pull-request under revision control
 - Preparing to run an experiment with a new version of the model
 - ...

Taking It Further

- Exercises #7 (cultural diffusion model) and #8 (harvester model) in Chapter 6 are great learning opportunities.